

1. How to add a new node at the beginning of a linked list.

head $\rightarrow$ [2] $\rightarrow$ [3] $\rightarrow$ [4] $\rightarrow$ NULL

New node $\rightarrow$ [1] $\rightarrow$ NULL

- Create a new Node
- enter the data
- Make the new node point to the current head
(new $\rightarrow$ next = head)
- update the head pointer (head = new)

Example!

Before!

Head $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ NULL

after insert 5.

Head $\rightarrow$ 5 $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ NULL

Logic!

1. Create a new node
 2. Store data
 3. new $\rightarrow$ next = head
 4. head = new
- Time complexity: O(1)

2. Find the length and display (print) the data.

To display the linked list:

```
temp = head
while (temp != null)
{
    print (temp->data);
    temp = temp->next;
}
```

2)

To find the length:

count = 0;

temp = head;

```

{ count = 1;
  temp = temp -> next;
}

```

Ex. Example:

Linked list:

Head $\rightarrow 5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$

display output:

5 10 20 30

length:

length = 4

Logic:

1. Start from the head node
2. visit each node one by one
3. Print the data
4. Increase the count by 1 for each node
5. stop when temp = NULL

Time complexity:

- Display: $O(n)$
- find length: $O(n)$

8. Display / Print the data.

```

void display()

```

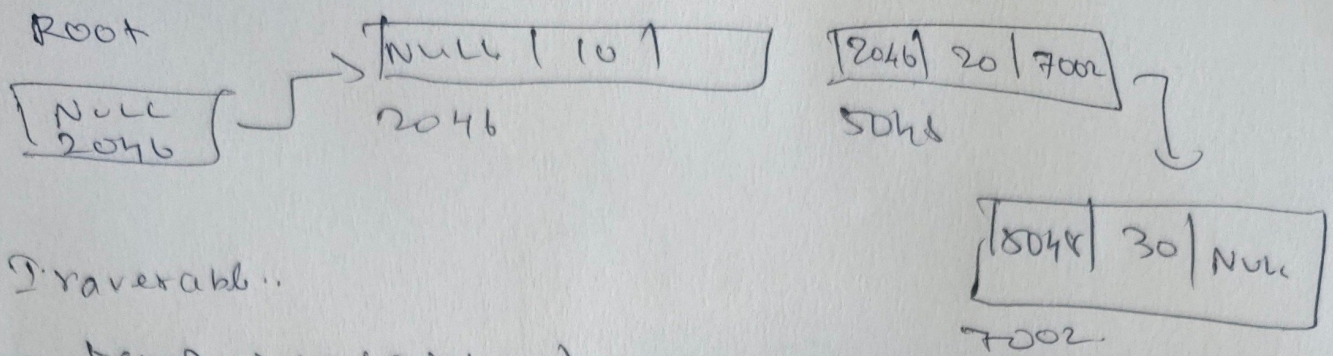
```

{ struct Node * temp = root;
  while (temp != NULL)
  { printf("%d -> ", temp->data);
    temp = temp -> next;
  }
}

```

algorithm:

1. start from head node.
2. if head == NULL, print "list is empty",
3. otherwise, move through each node
4. Print the data of each node.
- 5 - stop when temp == NULL



Traversable..

temp → 10 (print 10)
 temp → 20 (print 20)
 temp → 30 (print 30)
 temp → NULL (stop).

1. code:

```

void at the begin()
{
  struct Node * temp;
  temp = (struct Node *) malloc (size of struct);
  printf ("Enter node data ");
  scanf ("%d", &temp->data);
  temp->left = NULL;
  temp->right = NULL;
  if (root == NULL)
  {
    root = temp;
  }
  else
  {
    temp->right = root;
    root->left = temp;
    root = temp;
  }
}
  
```